



UNIVERSIDAD
SERGIO ARBOLEDA

Orbit Engine: Plataforma SaaS para la Gestión Integral de Procesos Internos en Pequeñas y Medianas Empresas

RINCÓN SUAREZ FABIÁN
RODRÍGUEZ FORERO NICOLÁS
VELASCO GONZÁLEZ DANIEL

UNIVERSIDAD SERGIO ARBOLEDA
ESCUELA DE CIENCIAS EXACTAS E INGENIERÍA
CIENCIAS DE LA COMPUTACIÓN E INTELIGENCIA ARTIFICIAL
BOGOTÁ D.C.
2026

Orbit Engine: Plataforma SaaS para la Gestión Integral de Procesos Internos en Pequeñas y Medianas Empresas

RINCÓN SUAREZ FABIÁN
RODRÍGUEZ FORERO NICOLÁS
VELASCO GONZÁLEZ DANIEL

ANEXOS

Asesor
Juan Pablo Ospina

Co-asesor
Camilo Rodríguez

UNIVERSIDAD SERGIO ARBOLEDA
ESCUELA DE CIENCIAS EXACTAS E INGENIERÍA
CIENCIAS DE LA COMPUTACIÓN E INTELIGENCIA ARTIFICIAL
BOGOTÁ D.C.
2026

Contenido

Lista de tablas	7
A. Anexo A. Manual de Usuario	9
1. Introducción	9
2. Acceso a la Plataforma	9
2.1. Página de inicio	9
2.2. Registro de organización	9
2.3. Invitación de usuarios	10
2.4. Inicio de sesión	10
2.5. Recuperación de contraseña	10
2.6. Cierre de sesión	10
3. Navegación General	10
4. Dashboard. Panel de Indicadores	11
5. Módulo de Inventario	11
5.1. Categorías	11
5.2. Productos	12
5.3. Ajuste de stock manual	13
5.4. Historial de movimientos	13
6. Módulo de Ventas	13
6.1. Registrar una venta	13
6.2. Ver detalle de una venta	14
6.3. Cancelar una venta	14
6.4. Filtros y búsqueda	14
7. Módulo de Clientes	15
7.1. Registrar un cliente	15
7.2. Editar un cliente	15
7.3. Desactivar un cliente	15
7.4. Historial de compras del cliente	15
8. Módulo de Administración	15
8.1. Ver usuarios	16
8.2. Crear un usuario	16
8.3. Editar un usuario	16
8.4. Eliminar un usuario	16
9. Configuración	16
9.1. Información personal	16
9.2. Cambio de contraseña	17

9.3.	Configuración de la organización	17
9.4.	Apariencia	17
9.5.	Eliminación de cuenta	17
10.	Preguntas Frecuentes	17
B. Anexo B. Manual de Instalación y Despliegue		19
1.	Visión General	19
2.	Prerrequisitos	19
2.1.	Herramientas requeridas	19
2.2.	Puertos utilizados	20
3.	Configuración de Variables de Entorno	20
4.	Entorno de Desarrollo Local	21
4.1.	Iniciar el entorno	22
4.2.	URLs del entorno local	22
4.3.	Detener el entorno	22
4.4.	Ver logs	22
5.	Migraciones de Base de Datos	22
5.1.	Aplicar migraciones pendientes	23
5.2.	Crear una nueva migración	23
5.3.	Ver historial de migraciones	23
5.4.	Revertir la última migración	23
5.5.	Migraciones del proyecto	23
6.	Pruebas	23
6.1.	Pruebas del backend (pytest)	23
6.2.	Pruebas del frontend (Playwright E2E)	24
7.	Generación del Cliente API	25
8.	Despliegue en Producción	25
8.1.	Prerrequisitos del servidor	25
8.2.	Preparar la red de Traefik	25
8.3.	Configurar Traefik con TLS	25
8.4.	Configurar variables de entorno para producción	26
8.5.	Construir y desplegar	26
8.6.	Actualización de la aplicación	26
9.	Monitoreo y Mantenimiento	27
9.1.	Estado de los servicios	27
9.2.	Backup de la base de datos	27
9.3.	Limpiar recursos no usados	27
10.	Solución de Problemas	27
C. Anexo C. Documentación Técnica		29
1.	Stack Tecnológico	29
1.1.	Backend	29
1.2.	Frontend	29

1.3.	Infraestructura	30
2.	Estructura del Proyecto	30
3.	Autenticación y Autorización	33
3.1.	Flujo de autenticación	33
3.2.	Control de acceso basado en roles (RBAC)	33
3.3.	Protección contra fuerza bruta	34
4.	API REST. Referencia de Endpoints	34
4.1.	Autenticación (/login)	34
4.2.	Usuarios (/users)	34
4.3.	Organizaciones (/organizations)	35
4.4.	Roles (/roles)	35
4.5.	Categorías (/categories)	35
4.6.	Productos (/products)	35
4.7.	Clientes (/customers)	36
4.8.	Ventas (/sales)	36
4.9.	Movimientos de inventario (/inventory-movements)	36
4.10.	Dashboard (/dashboard)	37
4.11.	Utilidades	37
5.	Modelos de Datos	38
5.1.	Organization	38
5.2.	Role	38
5.3.	User	38
5.4.	Category	39
5.5.	Product	39
5.6.	Customer	40
5.7.	Sale	40
5.8.	SaleItem	41
5.9.	InventoryMovement	41
6.	Variables de Entorno	42
6.1.	Variables requeridas	42
6.2.	Variables opcionales con valores por defecto	42
6.3.	Propiedades calculadas (no en .env)	43
7.	Convenciones de Código	43
7.1.	Backend (Python)	43
7.2.	Frontend (TypeScript/React)	44
8.	Pipeline de CI/CD	45
8.1.	Workflow de pruebas (en cada PR y push a main)	45
8.2.	Workflow de despliegue	45
9.	Estructura de Pruebas	46
9.1.	Backend (pytest)	46
9.2.	Frontend (Playwright E2E)	47
10.	Generación del Cliente API	47
11.	Multi-Tenancy	47



Lista de tablas

A.1. Navegación General	11
A.2. Historial de movimientos	13
B.1. Visión General	19
B.2. Herramientas requeridas	20
B.3. Puertos utilizados	20
B.4. URLs del entorno local	22
B.5. Migraciones del proyecto	23
B.6. Solución de Problemas	27
B.6. Solución de Problemas	28
C.1. Backend	29
C.2. Frontend	29
C.2. Frontend	30
C.3. Infraestructura	30
C.4. Flujo de autenticación	33
C.5. Control de acceso basado en roles (RBAC)	34
C.6. Autenticación (/login)	34
C.7. Usuarios (/users)	34
C.7. Usuarios (/users)	35
C.8. Organizaciones (/organizations)	35
C.9. Roles (/roles)	35
C.10. Categorías (/categories)	35
C.11. Productos (/products)	36
C.12. Clientes (/customers)	36
C.13. Ventas (/sales)	36
C.14. Movimientos de inventario (/inventory-movements)	37
C.15. Dashboard (/dashboard)	37
C.16. Utilidades	37
C.17. Variables requeridas	42
C.18. Variables opcionales con valores por defecto	42
C.18. Variables opcionales con valores por defecto	43
C.19. Propiedades calculadas (no en .env)	43
C.20. Backend (pytest)	46



UNIVERSIDAD
SERGIO ARBOLEDA



Anexo A. Manual de Usuario

OrbitEngine: Plataforma SaaS para la Gestión de Procesos Internos en Pymes
Versión: 1.0 | Abril 2026
Universidad Sergio Arboleda, Semillero de Software como Innovación

1. Introducción

Este manual describe el uso de OrbitEngine para los usuarios finales de la plataforma. OrbitEngine es una aplicación web multi-tenant que permite a pequeñas y medianas empresas gestionar en un solo lugar sus operaciones de **inventario**, **ventas** y **clientes**, con un panel de indicadores clave de desempeño (KPIs) accesible en tiempo real.

El sistema opera completamente desde el navegador web, sin necesidad de instalar software adicional. Cada empresa (organización) tiene su propio espacio de trabajo completamente aislado de las demás.

2. Acceso a la Plataforma

2.1. Página de inicio

Al ingresar a la URL de OrbitEngine, el usuario accede a la **página de presentación** (landing page) que muestra las características de la plataforma. Desde esta página se puede navegar a las opciones de inicio de sesión o registro.

2.2. Registro de organización

Para crear una nueva empresa en la plataforma:

1. Hacer clic en “**Comenzar gratis**” o en el enlace de registro en la barra de navegación.
2. En la pantalla `/signup-org`, completar el formulario con:
 - **Nombre de la organización:** nombre visible de la empresa.
 - **Slug:** identificador único en minúsculas (ej. `mi-empresa`). Solo letras, números y guiones; entre 3 y 50 caracteres.
 - **Descripción** (*opcional*): breve descripción del negocio.

3. Navegación General

- **Nombre, apellido y correo electrónico** del administrador.
 - **Contraseña** de al menos 8 caracteres.
3. Hacer clic en “**Crear organización**”.
 4. El sistema crea la organización y la cuenta administrador automáticamente y redirige al panel de control.

2.3. Invitación de usuarios

Los usuarios adicionales **no se registran por su cuenta**: son creados por el administrador desde el módulo **Admin**. Una vez creada la cuenta, el usuario recibe las credenciales y puede ingresar con ellas.

2.4. Inicio de sesión

1. Ir a la pantalla `/login`.
2. Ingresar el **correo electrónico** y la **contraseña**.
3. Hacer clic en “**Ingresar**”.
4. Si las credenciales son correctas, el sistema redirige al **panel de control** (`/dashboard`).

2.5. Recuperación de contraseña

Si olvidó su contraseña:

1. En la pantalla de inicio de sesión, hacer clic en “**¿Olvidaste tu contraseña?**”.
2. Ingresar el correo electrónico asociado a la cuenta.
3. Revisar la bandeja de entrada y hacer clic en el enlace recibido.
4. En la pantalla `/reset-password`, ingresar y confirmar la nueva contraseña.
5. Hacer clic en “**Restablecer contraseña**”.

2.6. Cierre de sesión

Para cerrar sesión, hacer clic en el **ícono de usuario** en la barra lateral izquierda y seleccionar “**Cerrar sesión**”. La sesión expira automáticamente después de 8 días de inactividad.

3. Navegación General

Una vez dentro del sistema, la interfaz se divide en:

- **Barra lateral izquierda**: menú principal de navegación con acceso a todos los módulos.



- **Área de contenido principal:** donde se muestra y gestiona la información de cada módulo.
- **Panel de usuario** (parte inferior de la barra lateral): acceso a configuración personal y cierre de sesión.

Los módulos disponibles son:

Cuadro A.1: Navegación General

Módulo	Ruta	Acceso
Dashboard	/dashboard	Todos los usuarios
Inventario	/dashboard/inventory	Todos los usuarios
Ventas	/dashboard/sales	Todos los usuarios
Clientes	/dashboard/customers	Todos los usuarios
Administración	/dashboard/admin	Solo Administradores
Configuración	/dashboard/settings	Todos los usuarios

4. Dashboard. Panel de Indicadores

El dashboard muestra un resumen del estado del negocio en tiempo real para la organización activa.

Indicadores disponibles:

- **Ventas del día:** total en dinero de las ventas registradas hoy.
- **Ventas del mes:** total acumulado del mes en curso.
- **Ticket promedio:** valor promedio por venta en el período.
- **Productos con stock bajo:** conteo de productos que están por debajo del mínimo configurado.
- **Top productos:** listado de los productos más vendidos.
- **Ventas por día:** gráfica de barras con la evolución diaria de ventas.

El dashboard se actualiza automáticamente con cada consulta. No requiere acción adicional del usuario.

5. Módulo de Inventario

El módulo de inventario permite gestionar los productos y categorías de la empresa, así como llevar el control detallado de los movimientos de stock.

5.1. Categorías

Las categorías permiten organizar el inventario en grupos lógicos. Admiten **jerarquía** (una categoría puede tener subcategorías).

Crear una categoría:

5. Módulo de Inventario

1. En el módulo Inventario, ir a la pestaña **Categorías**.
2. Hacer clic en **“Agregar categoría”**.
3. Completar el **nombre** (*requerido*) y opcionalmente una **descripción** y una **categoría padre**.
4. Guardar.

Editar o desactivar una categoría:

- Hacer clic en el menú de acciones (:) de la fila y seleccionar **“Editar”** o **“Desactivar”**.

5.2. Productos

Crear un producto:

1. En el módulo Inventario, ir a la pestaña **Productos**.
2. Hacer clic en **“Agregar producto”**.
3. Completar el formulario:
 - **Nombre** (*requerido*)
 - **SKU**: código único del producto (*requerido*)
 - **Categoría** (*opcional*)
 - **Precio de costo y precio de venta**
 - **Stock actual, stock mínimo y stock máximo** (*opcional*)
 - **Unidad de medida** (ej. unit, kg, lt)
 - **Código de barras** (*opcional*)
 - **Descripción e imagen** (*opcionales*)
4. Guardar.

Editar un producto:

- Hacer clic en el menú de acciones (:) → **“Editar”**. Todos los campos son modificables excepto el SKU si ya tiene movimientos.

Desactivar un producto:

- Hacer clic en el menú de acciones (:) → **“Desactivar”**. El producto deja de aparecer en los formularios de venta pero su historial se conserva.

Alerta de stock bajo:

- Los productos con $\text{stock_quantity} \leq \text{stock_min}$ aparecen resaltados en la tabla y se contabilizan en el indicador del dashboard.

5.3. Ajuste de stock manual

Para corregir el inventario por diferencias de conteo, mermas u otros motivos:

1. En la tabla de productos, seleccionar el producto y hacer clic en “**Ajustar stock**”.
2. Ingresar la **cantidad** (positiva para agregar, negativa para restar).
3. Ingresar un **motivo** del ajuste (*requerido*).
4. Confirmar.

El ajuste queda registrado en el historial de movimientos con el tipo **adjustment**.

5.4. Historial de movimientos

El historial registra todos los cambios de stock de cada producto:

Cuadro A.2: Historial de movimientos

Tipo de movimiento	Descripción
sale	Salida generada por una venta
adjustment	Entrada o salida manual
return	Entrada por devolución o cancelación de venta
purchase	Entrada por compra

Para ver el historial de un producto:

1. En la tabla de productos, hacer clic en el menú de acciones (:) → “**Ver historial de movimientos**”.
2. Se muestra la tabla con fecha, tipo, cantidad, stock anterior y stock resultante, y el usuario que realizó el movimiento.

6. Módulo de Ventas

El módulo de ventas permite registrar transacciones comerciales, visualizar el historial y cancelar ventas cuando sea necesario.

6.1. Registrar una venta

1. En el módulo Ventas, hacer clic en “**Nueva venta**”.
2. Completar el formulario:
 - **Cliente** (*opcional*): buscar por nombre o documento.
 - **Método de pago**: Efectivo, Tarjeta, Transferencia u Otro.
 - **Descuento y IVA/impuesto** (*opcional, en valores monetarios*).
 - **Notas** (*opcional*).

6. Módulo de Ventas

3. En la sección de **ítems**, agregar los productos de la venta:
 - Buscar el producto por nombre o SKU.
 - Ingresar la **cantidad**.
 - El sistema calcula el precio unitario y el subtotal del ítem automáticamente.
4. El sistema calcula en tiempo real: subtotal, descuento, impuesto y **total**.
5. Hacer clic en **“Registrar venta”**.

Al guardar, el sistema:

- Genera un **número de factura** único.
- Descuenta las cantidades del stock de cada producto registrado.
- Registra los movimientos de inventario correspondientes (**sale**).
- Actualiza el total de compras del cliente, si se asoció uno.

6.2. Ver detalle de una venta

En la tabla de ventas, hacer clic en el menú de acciones (:) → **“Ver detalle”**. Se muestra la información completa: número de factura, cliente, fecha, ítems, totales y método de pago.

6.3. Cancelar una venta

Una venta puede cancelarse mientras tenga estado **completed**:

1. En la tabla de ventas, hacer clic en el menú de acciones (:) → **“Cancelar venta”**.
2. Ingresar el **motivo de cancelación** (*requerido*).
3. Confirmar.

Al cancelar, el sistema:

- Cambia el estado de la venta a **cancelled**.
- **Revierte el stock** de cada producto afectado (movimiento tipo **return**).
- Registra la fecha y usuario de cancelación.

6.4. Filtros y búsqueda

La tabla de ventas permite:

- Buscar por **número de factura**.
- Filtrar por **estado** (**completed**, **cancelled**) y **método de pago**.
- Ordenar por cualquier columna.

7. Módulo de Clientes

El módulo permite gestionar el directorio de clientes de la organización y consultar su historial de compras.

7.1. Registrar un cliente

1. En el módulo Clientes, hacer clic en **“Agregar cliente”**.
2. Completar el formulario:
 - **Tipo de documento:** CC, NIT, Pasaporte, Otro (*requerido*).
 - **Número de documento** (*requerido, único por organización*).
 - **Nombre y apellido** (*requeridos*).
 - **Correo electrónico, teléfono, dirección, ciudad y país** (*opcionales*).
 - **Notas internas** (*opcionales*).
3. Guardar.

7.2. Editar un cliente

- Hacer clic en el menú de acciones (:) → **“Editar”**. Se pueden modificar todos los campos excepto el número de documento.

7.3. Desactivar un cliente

- Hacer clic en el menú de acciones (:) → **“Desactivar”**. El cliente se desactiva pero su historial de compras se conserva.

7.4. Historial de compras del cliente

- Hacer clic en el menú de acciones (:) → **“Ver historial de compras”**.
- Se muestran todas las ventas asociadas al cliente con fecha, número de factura, total y estado.
- Se visualizan también los indicadores: **total acumulado de compras, número de compras y fecha de última compra**.

8. Módulo de Administración

Este módulo es exclusivo para usuarios con el rol **Administrador**.

El módulo de administración permite gestionar las cuentas de usuario de la organización.

9. Configuración

8.1. Ver usuarios

La tabla muestra todos los usuarios de la organización con su nombre, correo, rol y estado (activo/inactivo).

8.2. Crear un usuario

1. Hacer clic en “**Agregar usuario**”.
2. Completar el formulario:
 - **Nombre y apellido.**
 - **Correo electrónico:** será el identificador de ingreso.
 - **Contraseña** temporal.
 - **Rol:** seleccionar entre los roles disponibles de la organización.
3. Guardar.

El nuevo usuario puede ingresar inmediatamente con las credenciales proporcionadas y cambiar su contraseña desde la configuración.

8.3. Editar un usuario

- Hacer clic en el menú de acciones (:) → “**Editar**”. Se pueden cambiar nombre, apellido y rol.

8.4. Eliminar un usuario

- Hacer clic en el menú de acciones (:) → “**Eliminar**”.
- Confirmar la acción en el diálogo de confirmación.

Advertencia: Esta acción es permanente. El historial de ventas y movimientos del usuario permanece registrado.

9. Configuración

El módulo de configuración permite al usuario gestionar su cuenta personal y, si es administrador, los datos de la organización.

9.1. Información personal

En la pestaña “**Perfil**”:

- Ver y editar **nombre y apellido**.
- Ver el **correo electrónico** asociado (no editable desde aquí).



9.2. Cambio de contraseña

En la pestaña “Seguridad”:

1. Ingresar la **contraseña actual**.
2. Ingresar la **nueva contraseña** y confirmarla.
3. Hacer clic en “**Actualizar contraseña**”.

9.3. Configuración de la organización

Disponible solo para usuarios con rol **Administrador**.

En la pestaña “Organización”:

- Editar el **nombre** de la organización.
- Editar el **slug** (*identificador único*).
- Editar la **descripción** y la **URL del logo**.

9.4. Apariencia

En la pestaña “Apariencia”:

- Cambiar entre **modo claro** y **modo oscuro**.

9.5. Eliminación de cuenta

En la pestaña “Cuenta”:

- El usuario puede solicitar la **eliminación de su propia cuenta**.
- Se requiere confirmación escribiendo el correo electrónico.

Nota: La eliminación de cuenta no elimina el historial de transacciones asociadas.

10. Preguntas Frecuentes

¿Puedo usar OrbitEngine desde el celular?

Sí. La interfaz está diseñada de manera responsiva y funciona en navegadores móviles modernos (Chrome, Safari). La experiencia óptima es en pantallas de escritorio o tabletas.

¿Mis datos están separados de los de otras empresas?

Sí. Cada organización tiene un espacio de trabajo completamente aislado. No es posible ver ni acceder a datos de otras organizaciones.

¿Qué pasa si cancelo una venta? ¿El stock vuelve?

Sí. Al cancelar una venta, el sistema revierte automáticamente el stock de todos los productos incluidos en ella.

¿Puedo tener múltiples roles para usuarios?

Actualmente cada usuario tiene un único rol. Los roles definen los permisos de acceso a los módulos y acciones disponibles.

¿Cómo sé cuándo un producto está por agotarse?

Los productos cuyo stock actual es igual o menor al **stock mínimo** configurado aparecen resaltados en la tabla de inventario y se contabilizan en el indicador “Productos con stock bajo” del dashboard.

¿Se puede exportar la información a Excel o PDF?

Los listados de inventario, ventas y clientes pueden exportarse a Excel (.xlsx) desde el módulo correspondiente, conservando los filtros aplicados. La exportación a PDF no está disponible en esta versión.

¿Qué navegadores son compatibles?

OrbitEngine es compatible con las versiones recientes de Chrome, Firefox, Edge y Safari. No se garantiza compatibilidad con Internet Explorer.

Documento generado como parte del proyecto de grado. Universidad Sergio Arboleda, Semillero de Software como Innovación, Abril 2026.

Anexo B. Manual de Instalación y Despliegue

OrbitEngine: Plataforma SaaS para la Gestión de Procesos Internos en Pymes
 Versión: 1.0 | Abril 2026
 Universidad Sergio Arboleda, Semillero de Software como Innovación

1. Visión General

OrbitEngine utiliza **Docker Compose** como orquestador de contenedores en todos los entornos. La arquitectura de despliegue se compone de los siguientes servicios:

Cuadro B.1: Visión General

Servicio	Imagen	Descripción
db	postgres:18	Base de datos relacional PostgreSQL
backend	Custom (FastAPI)	API REST del sistema
frontend	Custom (React/Nginx)	Interfaz de usuario compilada
prestart	Custom (FastAPI)	Ejecuta migraciones y datos iniciales al iniciar
adminer	adminer	Panel web de administración de BD
proxy	traefik:3.6	Reverse proxy y gestor de rutas HTTP/HTTPS
mailcatcher	schickling/mailcatcher	Servidor SMTP de prueba (solo dev)

El enrutamiento externo se gestiona con **Traefik**, que expone los servicios bajo subdominios configurados y maneja la terminación TLS automática mediante Let's Encrypt en producción.

2. Prerrequisitos

2.1. Herramientas requeridas

3. Configuración de Variables de Entorno

Cuadro B.2: Herramientas requeridas

Herramienta	Versión mínima	Propósito
Docker Engine	24.x	Contenedores
Docker	v2.x	Orquestación de servicios
Compose		
Git	2.x	Control de versiones
uv	0.4+	Gestor de dependencias Python (para desarrollo local sin Docker)
Bun	1.x	Runtime JS / gestor de paquetes (para desarrollo local sin Docker)

2.2. Puertos utilizados

Cuadro B.3: Puertos utilizados

Puerto	Servicio	Entorno
80	Traefik HTTP	Dev y Producción
443	Traefik HTTPS	Producción
8000	Backend FastAPI	Dev (expuesto directamente)
5173	Frontend (Vite)	Dev
5432	PostgreSQL	Dev
8080	Adminer	Dev
8090	Traefik Dashboard	Dev
1025	Mailcatcher SMTP	Dev
1080	Mailcatcher UI	Dev

3. Configuración de Variables de Entorno

Antes de ejecutar cualquier entorno, crear el archivo `.env` en la raíz del repositorio:

```
cp .env.example .env # si existe el archivo de ejemplo, de lo  
→ contrario crearlo manualmente
```

Las variables requeridas son:

```
# — Proyecto  
→  
PROJECT_NAME=OrbitEngine  
STACK_NAME=orbitengine-stack  
DOMAIN=localhost # En producción: tu-dominio.com  
  
# — Backend  
→  
SECRET_KEY=cambia-esto-en-produccion # Clave aleatoria, mínimo 32  
→ chars
```

```

BACKEND_CORS_ORIGINS=["http://localhost:5173","http://localhost"]
FRONTEND_HOST=http://localhost:5173    # En producción:
→ https://tu-dominio.com

# — Base de datos
→
POSTGRES_SERVER=db
POSTGRES_PORT=5432
POSTGRES_USER=postgres
POSTGRES_PASSWORD=cambia-esto
POSTGRES_DB=app

# — Superusuario inicial
→
FIRST_SUPERUSER=admin@ejemplo.com
FIRST_SUPERUSER_PASSWORD=cambia-esto

# — Email
→
SMTP_HOST=mailcatcher                # En producción:
→ smtp.tu-proveedor.com
SMTP_PORT=1025                        # En producción: 587
SMTP_TLS=false                        # En producción: true
EMAILS_FROM_EMAIL=noreply@tu-dominio.com

# — Imágenes Docker
→
DOCKER_IMAGE_BACKEND=backend
DOCKER_IMAGE_FRONTEND=frontend
ENVIRONMENT=local                    # local | staging | production

```

Producción: Cambiar todas las claves y contraseñas por valores aleatorios seguros. Se recomienda gestionar las variables de entorno directamente en el panel de Railway (backend) y en el panel de Vercel (frontend), evitando archivos `.env` en producción.

4. Entorno de Desarrollo Local

El entorno de desarrollo combina `compose.yml` con `compose.override.yml`, que añade los servicios de desarrollo (Traefik local, Mailcatcher) y sobrescribe las configuraciones para recarga en caliente.

5. Migraciones de Base de Datos

4.1. Iniciar el entorno

Desde la raíz del repositorio

```
docker compose watch
```

Este comando:

1. Construye las imágenes de backend y frontend si no existen.
2. Levanta todos los servicios en orden de dependencias.
3. Activa la recarga automática del backend al detectar cambios en `./backend`.
4. El servicio **prestart** ejecuta las migraciones de Alembic y los datos iniciales antes de que el backend acepte peticiones.

4.2. URLs del entorno local

Cuadro B.4: URLs del entorno local

Servicio	URL
Frontend	http://localhost:5173
Backend API	http://localhost:8000
Documentación interactiva (Swagger)	http://localhost:8000/docs
ReDoc	http://localhost:8000/redoc
Adminer	http://localhost:8080
Traefik Dashboard	http://localhost:8090
Mailcatcher UI	http://localhost:1080

4.3. Detener el entorno

Detener y eliminar contenedores (datos persisten en volumen)

```
docker compose down
```

Detener y eliminar contenedores + volúmenes (elimina datos de BD)

```
docker compose down -v
```

4.4. Ver logs

```
docker compose logs -f backend    # Logs del backend en tiempo real
```

```
docker compose logs -f frontend    # Logs del frontend
```

```
docker compose logs -f db          # Logs de PostgreSQL
```

5. Migraciones de Base de Datos

Las migraciones se gestionan con **Alembic** y se ejecutan automáticamente al iniciar el entorno (servicio **prestart**). Para operaciones manuales:

5.1. Aplicar migraciones pendientes

```
docker compose exec backend alembic upgrade head
```

5.2. Crear una nueva migración

Después de modificar los modelos en `backend/app/models.py`:

```
docker compose exec backend alembic revision --autogenerate -m
↪ "descripcion_del_cambio"
```

Revisar el archivo generado en `backend/app/alembic/versions/` antes de aplicarlo.

5.3. Ver historial de migraciones

```
docker compose exec backend alembic history --verbose
```

5.4. Revertir la última migración

```
docker compose exec backend alembic downgrade -1
```

5.5. Migraciones del proyecto

Las migraciones actuales del esquema son:

Cuadro B.5: Migraciones del proyecto

Archivo	Descripción
001_initial_schema.py	Tablas base: <code>organizations</code> , <code>roles</code> , <code>users</code>
002_categories.py	Tabla <code>categories</code> con jerarquía padre-hijo
003_products.py	Tabla <code>products</code>
004_customers.py	Tabla <code>customers</code>
005_inventory_movements.py	Tabla <code>inventory_movements</code>
006_sales.py	Tablas <code>sales</code> y <code>sale_items</code>

6. Pruebas

6.1. Pruebas del backend (pytest)

```
# Ejecutar todas las pruebas con cobertura
cd backend
uv run bash scripts/test.sh
```

```
# Prueba de un archivo específico
```



6. Pruebas

```
uv run pytest tests/api/routes/test_sales.py -v

# Prueba de una función específica
uv run pytest tests/api/routes/test_users.py::test_get_users_superuser_me
→ -v

# Pruebas por patrón
uv run pytest -k "test_create" -v

# Reporte de cobertura
uv run coverage run -m pytest && coverage report
```

La cobertura de pruebas abarca los principales módulos del sistema, tanto a nivel de API como de lógica CRUD. Se realizaron pruebas completas de API y CRUD para los módulos de usuarios, categorías, productos, clientes, ventas, movimientos de inventario y dashboard, lo que garantiza la validez funcional y la robustez de estos componentes. En el módulo de autenticación, la cobertura se concentra exclusivamente en pruebas de la API, debido a que la lógica principal reside en los flujos de autenticación y autorización más que en operaciones CRUD tradicionales. Esta cobertura integral asegura que los procesos críticos relacionados con la gestión de datos y la seguridad hayan sido evaluados mediante casos de prueba automatizados.

6.2. Pruebas del frontend (Playwright E2E)

```
cd frontend

# Ejecutar todas las pruebas E2E
bun run test

# Prueba de un archivo específico
bunx playwright test tests/login.spec.ts

# Prueba por descripción
bunx playwright test --grep "crear venta"

# Modo con interfaz visual
bun run test:ui
```

Las pruebas E2E cubren los flujos: login, registro de organización, registro de usuario, restablecimiento de contraseña, inventario, ventas, clientes, administración de usuarios y configuración.

7. Generación del Cliente API

Cuando se modifican los endpoints del backend, el cliente TypeScript del frontend debe regenerarse:

```
# El backend debe estar corriendo en http://localhost:8000
cd frontend
bun run generate-client
```

Esto actualiza los archivos en `src/client/` a partir del esquema OpenAPI del backend. **No editar manualmente** estos archivos.

8. Despliegue en Producción

El despliegue en producción usa el mismo `compose.yml` sin el `compose.override.yml`.

8.1. Prerrequisitos del servidor

- Servidor Linux con Docker Engine y Docker Compose instalados.
- Dominio apuntando a la IP del servidor (registros A/CNAME en el DNS).
- Puertos 80 y 443 abiertos en el firewall.

8.2. Preparar la red de Traefik

Traefik requiere una red Docker externa compartida entre todos los servicios:

```
docker network create traefik-public
```

8.3. Configurar Traefik con TLS

Crear el archivo de configuración de Traefik (`compose.traefik.yml` ya incluido en el repositorio) y levantarlo:

```
docker compose -f compose.traefik.yml up -d
```

Esto levanta Traefik en modo producción con:

- Redirección automática HTTP → HTTPS.
- Certificados TLS gestionados por Let's Encrypt.
- Panel de Traefik accesible solo internamente.

8.4. Configurar variables de entorno para producción

Editar el `.env` con los valores de producción:

```
DOMAIN=tu-dominio.com
ENVIRONMENT=production
SECRET_KEY=<clave-aleatoria-larga>
POSTGRES_PASSWORD=<contraseña-segura>
FIRST_SUPERUSER_PASSWORD=<contraseña-segura>
FRONTEND_HOST=https://tu-dominio.com
BACKEND_CORS_ORIGINS=["https://tu-dominio.com"]
SMTP_HOST=smtp.tu-proveedor.com
SMTP_PORT=587
SMTP_TLS=true
EMAILS_FROM_EMAIL=noreply@tu-dominio.com
```

8.5. Construir y desplegar

```
# Construir imágenes con la etiqueta de versión
TAG=v1.0.0 docker compose build
```

```
# Iniciar todos los servicios
TAG=v1.0.0 docker compose up -d
```

```
# Verificar estado
docker compose ps
docker compose logs -f
```

Los servicios se levantan en este orden:

1. db: espera estar healthy (hasta 30s).
2. prestart: ejecuta migraciones y datos iniciales.
3. backend y frontend: arrancan tras prestart.
4. adminer: panel de administración de BD.

8.6. Actualización de la aplicación

```
# Obtener la nueva versión
git pull origin main
```

```
# Reconstruir imágenes
TAG=v1.1.0 docker compose build
```

```
# Reiniciar servicios (downtime mínimo: solo el backend se reinicia)
```

```
TAG=v1.1.0 docker compose up -d --no-deps backend frontend
```

Las migraciones se aplican automáticamente al iniciar prestart

9. Monitoreo y Mantenimiento

9.1. Estado de los servicios

```
docker compose ps                # Estado de todos los
→ contenedores
docker stats                      # Uso de CPU/RAM en tiempo real
docker compose exec db psql -U postgres -c "\l" # Listar BDs en
→ PostgreSQL
```

9.2. Backup de la base de datos

Crear un backup

```
docker compose exec db pg_dump -U postgres app > backup_$(date
→ +%Y%m%d).sql
```

Restaurar un backup

```
docker compose exec -T db psql -U postgres app < backup_20260401.sql
```

9.3. Limpiar recursos no usados

```
docker system prune -f           # Elimina imágenes, contenedores y
→ redes sin usar
docker volume prune -f          # Elimina volúmenes sin usar
→ (¡precaución en producción!)
```

10. Solución de Problemas

Cuadro B.6: Solución de Problemas

Problema	Causa probable	Solución
El backend no inicia	BD no está disponible	Verificar que db esté healthy con <code>docker compose ps</code> . Revisar <code>POSTGRES_*</code> en <code>.env</code> .
Error de migraciones al iniciar	Migración incompatible	Ejecutar <code>docker compose exec backend alembic history</code> y <code>alembic current</code> . Corregir el script de migración.
502 Bad Gateway en el frontend	Backend no responde	Verificar logs del backend: <code>docker compose logs backend</code> . Comprobar el healthcheck.
Emails no se envían (producción)	SMTP mal configurado	Verificar <code>SMTP_HOST</code> , <code>SMTP_PORT</code> y credenciales. Probar con <code>telnet \$SMTP_HOST \$SMTP_PORT</code> .

10. Solución de Problemas

Cuadro B.6: Solución de Problemas

Problema	Causa probable	Solución
<code>Secret key too short</code> al iniciar	<code>SECRET_KEY</code> es demasiado corta	Generar una clave con <code>openssl rand -hex 32</code> y actualizar <code>.env</code> .
Traefik no renueva el certificado TLS	Puerto 80 bloqueado o DNS incorrecto	Verificar que el dominio resuelva a la IP correcta y el puerto 80 esté abierto.
Error <code>UNIQUE constraint</code> al insertar	Dato duplicado en BD	Verificar que el SKU, slug u otro campo único no esté repetido.

Documento generado como parte del proyecto de grado. Universidad Sergio Arboleda, Semillero de Software como Innovación, Abril 2026.

Anexo C. Documentación Técnica

OrbitEngine: Plataforma SaaS para la Gestión de Procesos Internos en Pymes
Versión: 1.0 | Abril 2026
Universidad Sergio Arboleda, Semillero de Software como Innovación

1. Stack Tecnológico

1.1. Backend

Cuadro C.1: Backend

Componente	Tecnología	Versión	Propósito
Lenguaje	Python	3.10+	Lenguaje principal del backend
Framework web	FastAPI	0.115+	API REST asíncrona con tipado estricto
ORM + Schemas	SQLModel	0.0.21+	Modelos compartidos entre SQLAlchemy y Pydantic
Motor de BD	SQLAlchemy	2.x	Capa de abstracción de base de datos
Driver PostgreSQL	Psycopg3	3.x	Conector PostgreSQL para Python
Migraciones	Alembic	1.x	Gestión de migraciones de esquema
Autenticación	PyJWT	2.x	Generación y verificación de tokens JWT
Hash de contraseñas	Pwplib	0.2+	Hashing seguro con bcrypt
Validación	Pydantic	2.x	Schemas de entrada/salida de datos
Configuración	Pydantic-Settings	2.x	Variables de entorno tipadas
Lintor/Formatter	Ruff	0.6+	Análisis estático y formato de código
Verificación de tipos	Mypy	1.x	Chequeo de tipos estático en modo estricto
Testing	Pytest + Coverage	8.x	Pruebas unitarias e integración
Monitoring	Sentry (opcional)	N/A	Rastreo de errores en producción

1.2. Frontend

Cuadro C.2: Frontend

Componente	Tecnología	Versión	Propósito
Lenguaje	TypeScript	5.x	Tipado estático sobre JavaScript
Framework UI	React	19.x	Librería de componentes reactivos
Build tool	Vite	6.x	Empaquetado y servidor de desarrollo

2. Estructura del Proyecto

Cuadro C.2: Frontend

Componente	Tecnología	Versión	Propósito
Router	TanStack Router	1.x	Enrutamiento con tipado por archivo
Server state	TanStack Query	5.x	Cache y sincronización de datos del servidor
Tablas	TanStack Table	8.x	Tablas con sorting/filtering/paginación
Formularios	React Hook Form	7.x	Manejo de formularios con rendimiento optimizado
Validación	Zod	3.x	Schemas de validación con inferencia de tipos
UI Components	Shadcn/ui	N/A	Componentes base sobre Radix UI
Estilos	Tailwind CSS	4.x	Utilidades CSS de bajo nivel
Gráficos	Recharts	2.x	Gráficas para el dashboard
HTTP Client	Axios (generado)	N/A	Cliente OpenAPI auto-generado
Notificaciones	Sonner	1.x	Toasts y notificaciones
Íconos	Lucide React	0.4+	Librería de íconos SVG
Linter/Formatter	Biome	1.x	Análisis estático y formato
E2E Testing	Playwright	1.x	Pruebas end-to-end en navegador

1.3. Infraestructura

Cuadro C.3: Infraestructura

Componente	Tecnología	Propósito
Base de datos	PostgreSQL 18	Almacenamiento relacional principal
Contenedores	Docker Compose v2	Orquestación de servicios
Reverse proxy	Traefik 3.6	Enrutamiento HTTP/HTTPS + TLS automático
CI/CD	GitHub Actions	Pipeline de integración y entrega continua
Panel de BD	Adminer	Administración web de PostgreSQL
Email (dev)	Mailcatcher	Interceptor SMTP para desarrollo

2. Estructura del Proyecto

```
orbit-engine/
├── .env                      # Variables de entorno (no incluir en git)
├── compose.yml              # Configuración Docker Compose base
├── compose.override.yml     # Sobreescrituras para desarrollo local
├── compose.traefik.yml      # Traefik para producción
├── pyproject.toml           # Configuración Ruff y Mypy (nivel raíz)
├── backend/
│   ├── Dockerfile
│   ├── pyproject.toml      # Dependencias Python (uv)
│   └── scripts/
│       ├── prestart.sh     # Ejecutado por el servicio prestart
│       ├── test.sh         # Script de pruebas con cobertura
│       └── lint.sh         # Mypy + Ruff check
```

```

├── format.sh          # Ruff format
├── app/
│   ├── main.py        # Punto de entrada FastAPI + registro de routers
│   ├── models.py      # Todos los modelos SQLAlchemy + schemas Pydantic
│   ├── crud.py        # Operaciones de base de datos
│   ├── utils.py       # Helpers (email, generación de tokens)
│   ├── backend_pre_start.py # Script de inicialización
│   ├── initial_data.py # Seed de datos iniciales (roles, superusuario)
│   └── api/
│       ├── deps.py    # Dependencias FastAPI (auth, sesión, roles)
│       └── routes/
│           ├── login.py      # Autenticación y recuperación de contraseña
│           ├── users.py     # Gestión de usuarios
│           ├── organizations.py # Registro y configuración de organización
│           ├── roles.py     # Listado de roles
│           ├── categories.py # CRUD de categorías
│           ├── products.py  # CRUD de productos + stock
│           ├── customers.py  # CRUD de clientes
│           ├── sales.py     # Registro y gestión de ventas
│           ├── inventory_movements.py # Historial de movimientos
│           ├── dashboard.py # Estadísticas del dashboard
│           ├── utils.py     # Health check
│           └── private.py   # Endpoints de superusuario
│   ├── core/
│       ├── config.py    # Settings con Pydantic-Settings
│       ├── db.py        # Engine SQLAlchemy
│       └── security.py  # JWT, hashing de contraseñas
│   └── alembic/
│       ├── env.py
│       └── versions/
│           ├── 001_initial_schema.py # organizations, roles, users
│           ├── 002_categories.py
│           ├── 003_products.py
│           ├── 004_customers.py
│           ├── 005_inventory_movements.py
│           └── 006_sales.py
├── frontend/
│   ├── Dockerfile
│   ├── package.json
│   ├── vite.config.ts
│   ├── openapi-ts.config.ts # Configuración del generador de cliente API
│   └── src/

```

2. Estructura del Proyecto

— main.tsx	# Punto de entrada React
— routeTree.gen.ts	# Árbol de rutas auto-generado (no editar)
— utils.ts	# Helpers compartidos
— client/	# Cliente API auto-generado (no editar)
— index.ts	
— sdk.gen.ts	# Funciones de llamada a la API
— types.gen.ts	# Tipos TypeScript desde OpenAPI
— schemas.gen.ts	
— core/	# Configuración Axios
— components/	
— Admin/	# Componentes de gestión de usuarios
— Common/	# Componentes reutilizables (DataTable, layouts)
— Customers/	# Componentes del módulo de clientes
— Inventory/	# Componentes del módulo de inventario
— Landing/	# Componentes de la página pública
— Pending/	# Componentes de estados de carga
— Sales/	# Componentes del módulo de ventas
— Sidebar/	# Navegación principal
— UserSettings/	# Configuración de perfil y organización
— theme-provider.tsx	
— ui/	# Componentes Shadcn/ui (no editar)
— hooks/	
— useAuth.ts	
— useCopyToClipboard.ts	
— useCustomToast.ts	
— useDebounce.ts	
— useMobile.ts	
— useScrollAnimation.ts	
— lib/	
— utils.ts	# cn() helper para Tailwind
— routes/	
— __root.tsx	# Layout raíz con providers
— index.tsx	# Landing page (/)
— login.tsx	# Inicio de sesión (/login)
— signup.tsx	# Registro de usuario (/signup)
— signup-org.tsx	# Registro de organización (/signup-org)
— recover-password.tsx	
— reset-password.tsx	
— dashboard/	
— index.tsx	# Dashboard KPIs (/dashboard)
— inventory.tsx	
— sales.tsx	
— customers.tsx	

```

├── admin.tsx
├── settings.tsx
├── docs/
│   └── informe-final/           # Documentación del proyecto de grado

```

3. Autenticación y Autorización

3.1. Flujo de autenticación

OrbitEngine usa **JWT Bearer Tokens** siguiendo el estándar OAuth2 Password Flow:

1. El cliente envía credenciales a `POST /api/v1/login/access-token`.
2. El backend valida las credenciales, verifica que la cuenta no esté bloqueada.
3. Si son válidas, retorna un `access_token` JWT.
4. El cliente incluye el token en el header `Authorization: Bearer <token>` en cada request.
5. El backend decodifica el token, extrae el `sub` (UUID del usuario) y carga el usuario de la BD.
6. La organización del usuario se extrae de `user.organization_id`, no del token directamente.

Parámetros del token:

Cuadro C.4: Flujo de autenticación

Parámetro	Valor
Algoritmo	HS256
Payload <code>sub</code>	UUID del usuario (<code>str</code>)
Expiración	8 días (11,520 minutos)
Clave de firma	<code>SECRET_KEY</code> del entorno

3.2. Control de acceso basado en roles (RBAC)

Los roles se almacenan en la tabla `roles` con permisos como array JSONB. La verificación en los endpoints usa la dependencia `require_role()`:

```

# Solo administradores pueden crear usuarios
@router.post("/", dependencies=[Depends(require_role("admin"))])
def create_user(...) -> Any: ...

```

```

# Administradores y vendedores pueden registrar ventas

```



4. API REST. Referencia de Endpoints

```
@router.post("/", dependencies=[Depends(require_role("admin",
→ "seller"))])
def create_sale(...) -> Any: ...
```

Dependencias de autenticación disponibles:

Cuadro C.5: Control de acceso basado en roles (RBAC)

Tipo anotado	Descripción
<code>CurrentUser</code>	Usuario autenticado; error 403 si token inválido
<code>CurrentAdminUser</code>	Usuario autenticado con rol <code>admin</code> ; error 403 si no
<code>CurrentOrganization</code>	UUID de organización extraído del <code>CurrentUser</code>
<code>SessionDep</code>	Sesión de base de datos <code>SQLModel</code>

3.3. Protección contra fuerza bruta

El modelo `User` incluye campos `failed_login_attempts` y `locked_until`. Tras `N` intentos fallidos consecutivos, la cuenta se bloquea temporalmente.

4. API REST. Referencia de Endpoints

Todos los endpoints están prefijados con `/api/v1`. La documentación interactiva se sirve en `/docs` (Swagger UI) y `/redoc`.

4.1. Autenticación (`/login`)

Cuadro C.6: Autenticación (`/login`)

Método	Ruta	Auth	Descripción
POST	<code>/login/access-token</code>	No	Obtener token JWT (login)
POST	<code>/login/test-token</code>	Sí	Verificar validez del token
POST	<code>/password-recovery/{email}</code>	No	Solicitar enlace de recuperación
POST	<code>/reset-password</code>	No	Restablecer contraseña con token

4.2. Usuarios (`/users`)

Cuadro C.7: Usuarios (`/users`)

Método	Ruta	Auth	Rol mínimo	Descripción
GET	<code>/users/</code>	Sí	admin	Listar usuarios de la organización
POST	<code>/users/</code>	Sí	admin	Crear usuario en la organización
GET	<code>/users/me</code>	Sí	Cualquiera	Obtener perfil propio

Cuadro C.7: Usuarios (/users)

Método	Ruta	Auth	Rol mínimo	Descripción
PATCH	/users/me	Sí	Cualquiera	Actualizar nombre/apellido/teléfono
DELETE	/users/me	Sí	Cualquiera	Eliminar cuenta propia (soft delete)
PATCH	/users/me/password	Sí	Cualquiera	Cambiar contraseña
GET	/users/{user_id}	Sí	admin	Obtener usuario por ID
PATCH	/users/{user_id}	Sí	admin	Actualizar usuario
DELETE	/users/{user_id}	Sí	admin	Eliminar usuario

4.3. Organizaciones (/organizations)

Cuadro C.8: Organizaciones (/organizations)

Método	Ruta	Auth	Rol mínimo	Descripción
GET	/organizations/me	Sí	Cualquiera	Ver datos de la organización actual
PATCH	/organizations/me	Sí	admin	Actualizar nombre/slug/descripción
POST	/organizations/signup	No	N/A	Registro de nueva organización + admin

4.4. Roles (/roles)

Cuadro C.9: Roles (/roles)

Método	Ruta	Auth	Descripción
GET	/roles/	Sí	Listar roles disponibles en la organización

4.5. Categorías (/categories)

Cuadro C.10: Categorías (/categories)

Método	Ruta	Auth	Rol mínimo	Descripción
GET	/categories/	Sí	Cualquiera	Listar categorías (soporta <code>search</code> , <code>is_active</code>)
POST	/categories/	Sí	admin	Crear categoría
GET	/categories/{id}	Sí	Cualquiera	Obtener categoría por ID
PATCH	/categories/{id}	Sí	admin	Actualizar categoría
DELETE	/categories/{id}	Sí	admin	Eliminar / desactivar categoría

4.6. Productos (/products)

4. API REST. Referencia de Endpoints

Cuadro C.11: Productos (/products)

Método	Ruta	Auth	Rol mínimo	Descripción
GET	/products/	Sí	Cualquiera	Listar productos (search, is_active, category_id, sort_by)
POST	/products/	Sí	admin	Crear producto
GET	/products/{id}	Sí	Cualquiera	Obtener producto por ID
PATCH	/products/{id}	Sí	admin	Actualizar producto
DELETE	/products/{id}	Sí	admin	Eliminar / desactivar producto
POST	/products/{id}/stock-adjust	Sí	admin	Ajuste manual de stock
GET	/products/{id}/movements	Sí	Cualquiera	Historial de movimientos del producto

4.7. Clientes (/customers)

Cuadro C.12: Clientes (/customers)

Método	Ruta	Auth	Rol mínimo	Descripción
GET	/customers/	Sí	Cualquiera	Listar clientes (search, is_active)
POST	/customers/	Sí	Cualquiera	Registrar cliente
GET	/customers/{id}	Sí	Cualquiera	Obtener cliente por ID
PATCH	/customers/{id}	Sí	Cualquiera	Actualizar cliente
DELETE	/customers/{id}	Sí	admin	Desactivar cliente
GET	/customers/{id}/sales	Sí	Cualquiera	Historial de compras del cliente

4.8. Ventas (/sales)

Cuadro C.13: Ventas (/sales)

Método	Ruta	Auth	Rol mínimo	Descripción
GET	/sales/	Sí	Cualquiera	Listar ventas (search, status, payment_method, sort_by)
POST	/sales/	Sí	Cualquiera	Registrar venta (descuenta stock automáticamente)
GET	/sales/{id}	Sí	Cualquiera	Obtener venta por ID
POST	/sales/{id}/cancel	Sí	admin	Cancelar venta (revierte stock)

4.9. Movimientos de inventario (/inventory-movements)

Cuadro C.14: Movimientos de inventario (/inventory-movements)

Método	Ruta	Auth	Descripción
GET	/inventory-movements/	Sí	Listar movimientos (<code>product_id</code> , <code>movement_type</code> , <code>sort_by</code>)

4.10. Dashboard (/dashboard)

Cuadro C.15: Dashboard (/dashboard)

Método	Ruta	Auth	Descripción
GET	/dashboard/stats	Sí	Estadísticas consolidadas de la organización

Respuesta de /dashboard/stats:

```
{
  "sales_today": { "count": 5, "total": "250000.00" },
  "sales_month": { "count": 42, "total": "1850000.00" },
  "low_stock_count": 3,
  "average_ticket": "44047.62",
  "top_products": [
    {
      "product_id": "...",
      "product_name": "Producto A",
      "quantity_sold": 15,
      "revenue": "450000.00"
    }
  ],
  "sales_by_day": [{ "date": "2026-04-01", "count": 3, "total":
    ↪ "120000.00" }]
}
```

4.11. Utilidades

Cuadro C.16: Utilidades

Método	Ruta	Auth	Descripción
GET	/utils/health-check/	No	Health check del servicio backend

5. Modelos de Datos

5.1. Organization

organizations

— id	UUID	PK
— name	VARCHAR(255)	NOT NULL
— slug	VARCHAR(100)	UNIQUE, INDEX
— description	TEXT	NULLABLE
— logo_url	VARCHAR(500)	NULLABLE
— is_active	BOOLEAN	DEFAULT true
— created_at	TIMESTAMPTZ	NOT NULL
— updated_at	TIMESTAMPTZ	NOT NULL

Índices: idx_organizations_slug (unique), idx_organizations_is_active

Validación del slug: expresión regular `^[a-z0-9-]{3,50}$` (solo minúsculas, números y guiones).

5.2. Role

roles

— id	INTEGER	PK (autoincrement)
— name	VARCHAR(50)	UNIQUE, INDEX
— description	TEXT	NULLABLE
— permissions	JSONB	DEFAULT []
— created_at	TIMESTAMPTZ	NOT NULL

Índices: idx_roles_name (unique)

5.3. User

users

— id	UUID	PK
— organization_id	UUID	FK → organizations.id, INDEX
— role_id	INTEGER	FK → roles.id, INDEX
— email	VARCHAR(255)	INDEX
— first_name	VARCHAR(100)	NOT NULL
— last_name	VARCHAR(100)	NOT NULL
— phone	VARCHAR(50)	NULLABLE
— avatar_url	VARCHAR(500)	NULLABLE
— hashed_password	VARCHAR(255)	NOT NULL
— is_active	BOOLEAN	DEFAULT true
— is_verified	BOOLEAN	DEFAULT false

— last_login_at	TIMESTAMPZ	NULLABLE
— failed_login_attempts	INTEGER	DEFAULT 0
— locked_until	TIMESTAMPZ	NULLABLE
— created_at	TIMESTAMPZ	NOT NULL
— updated_at	TIMESTAMPZ	NOT NULL
— deleted_at	TIMESTAMPZ	NULLABLE (soft delete)

Restricciones: UNIQUE(organization_id, email)

Índices: idx_users_organization_id, idx_users_role_id, idx_users_is_active

5.4. Category

categories

— id	UUID	PK
— organization_id	UUID	FK → organizations.id, INDEX
— parent_id	UUID	FK → categories.id, NULLABLE (jerarquía)
— name	VARCHAR(255)	NOT NULL
— description	TEXT	NULLABLE
— is_active	BOOLEAN	DEFAULT true
— created_at	TIMESTAMPZ	NOT NULL
— updated_at	TIMESTAMPZ	NOT NULL
— deleted_at	TIMESTAMPZ	NULLABLE (soft delete)

Restricciones: UNIQUE(organization_id, name, parent_id)

Índices: idx_categories_organization_id, idx_categories_parent_id, idx_categories_is_a

5.5. Product

products

— id	UUID	PK
— organization_id	UUID	FK → organizations.id, INDEX
— category_id	UUID	FK → categories.id, NULLABLE
— name	VARCHAR(255)	NOT NULL
— sku	VARCHAR(100)	NOT NULL
— description	TEXT	NULLABLE
— image_url	VARCHAR(500)	NULLABLE
— cost_price	NUMERIC(12,2)	DEFAULT 0
— sale_price	NUMERIC(12,2)	DEFAULT 0
— stock_quantity	INTEGER	DEFAULT 0
— stock_min	INTEGER	DEFAULT 0
— stock_max	INTEGER	NULLABLE
— unit	VARCHAR(50)	DEFAULT 'unit'
— barcode	VARCHAR(100)	NULLABLE

5. Modelos de Datos

— is_active	BOOLEAN	DEFAULT true
— created_at	TIMESTAMPTZ	NOT NULL
— updated_at	TIMESTAMPTZ	NOT NULL
— deleted_at	TIMESTAMPTZ	NULLABLE (soft delete)

Restricciones: UNIQUE(organization_id, sku)

Índices: idx_products_organization_id, idx_products_category_id, idx_products_is_active

Stock bajo: un producto está en alerta cuando $\text{stock_quantity} \leq \text{stock_min}$.

5.6. Customer

customers

— id	UUID	PK
— organization_id	UUID	FK → organizations.id, INDEX
— document_type	VARCHAR(50)	NOT NULL (CC, NIT, Pasaporte, Otro)
— document_number	VARCHAR(50)	NOT NULL
— first_name	VARCHAR(100)	NOT NULL
— last_name	VARCHAR(100)	NOT NULL
— email	VARCHAR(255)	NULLABLE, INDEX
— phone	VARCHAR(50)	NULLABLE, INDEX
— address	TEXT	NULLABLE
— city	VARCHAR(100)	NULLABLE
— country	VARCHAR(100)	NULLABLE
— notes	TEXT	NULLABLE
— is_active	BOOLEAN	DEFAULT true
— total_purchases	NUMERIC(12,2)	DEFAULT 0 (actualizado automáticamente)
— purchases_count	INTEGER	DEFAULT 0 (actualizado automáticamente)
— last_purchase_at	TIMESTAMPTZ	NULLABLE (actualizado automáticamente)
— created_at	TIMESTAMPTZ	NOT NULL
— updated_at	TIMESTAMPTZ	NOT NULL
— deleted_at	TIMESTAMPTZ	NULLABLE (soft delete)

Restricciones: UNIQUE(organization_id, document_number)

Índices: idx_customers_organization_id, idx_customers_email, idx_customers_phone, idx_

5.7. Sale

sales

— id	UUID	PK
— organization_id	UUID	FK → organizations.id, INDEX
— customer_id	UUID	FK → customers.id, NULLABLE
— user_id	UUID	FK → users.id, INDEX

— invoice_number	VARCHAR(100)	NOT NULL
— sale_date	TIMESTAMPTZ	NOT NULL
— subtotal	NUMERIC(12,2)	DEFAULT 0
— discount	NUMERIC(12,2)	DEFAULT 0
— tax	NUMERIC(12,2)	DEFAULT 0
— total	NUMERIC(12,2)	DEFAULT 0
— payment_method	VARCHAR(50)	DEFAULT 'cash' (cash/card/transfer/other)
— status	VARCHAR(50)	DEFAULT 'completed' (completed/cancelled/pendi
— notes	TEXT	NULLABLE
— cancelled_at	TIMESTAMPTZ	NULLABLE
— cancelled_by	UUID	FK → users.id, NULLABLE
— cancellation_reason	TEXT	NULLABLE
— created_at	TIMESTAMPTZ	NOT NULL
— updated_at	TIMESTAMPTZ	NOT NULL

Restricciones: UNIQUE(organization_id, invoice_number)

Índices: idx_sales_organization_id, idx_sales_customer_id, idx_sales_user_id,
idx_sales_status, idx_sales_sale_date, idx_sales_invoice_number

Fórmula: total = subtotal - discount + tax

5.8. SaleItem

sale_items

— id	UUID	PK
— sale_id	UUID	FK → sales.id, INDEX
— product_id	UUID	FK → products.id, INDEX
— product_name	VARCHAR(255)	NOT NULL (snapshot al momento de la venta)
— product_sku	VARCHAR(100)	NOT NULL (snapshot)
— quantity	INTEGER	NOT NULL
— unit_price	NUMERIC(12,2)	NOT NULL
— subtotal	NUMERIC(12,2)	NOT NULL
— created_at	TIMESTAMPTZ	NOT NULL

Índices: idx_sale_items_sale_id, idx_sale_items_product_id

Los campos `product_name` y `product_sku` son snapshots del producto en el momento de la venta. Si el producto se edita posteriormente, el historial de ventas conserva los datos originales.

5.9. InventoryMovement

inventory_movements

6. Variables de Entorno

id	UUID	PK
organization_id	UUID	FK → organizations.id, INDEX
product_id	UUID	FK → products.id, INDEX
user_id	UUID	FK → users.id, INDEX
movement_type	VARCHAR(50)	NOT NULL (sale/adjustment/return/purchase)
quantity	INTEGER	NOT NULL (positivo = entrada, negativo = salida)
previous_stock	INTEGER	NOT NULL
new_stock	INTEGER	NOT NULL
reference_id	UUID	NULLABLE (FK al objeto de origen)
reference_type	VARCHAR(50)	NULLABLE (sale/adjustment/return)
reason	TEXT	NULLABLE
created_at	TIMESTAMPTZ	NOT NULL

Índices: idx_inventory_movements_organization_id, idx_inventory_movements_product_id, idx_inventory_movements_user_id, idx_inventory_movements_movement_type, idx_inventory_movements_created_at, idx_inventory_movements_reference (reference_type, reference_id)

6. Variables de Entorno

La configuración se carga desde el archivo `.env` usando Pydantic-Settings. Todas las variables se validan al iniciar el servicio.

6.1. Variables requeridas

Cuadro C.17: Variables requeridas

Variable	Tipo	Descripción
PROJECT_NAME	str	Nombre del proyecto
SECRET_KEY	str	Clave para firmar los JWT
POSTGRES_SERVER	str	Host del servidor PostgreSQL
POSTGRES_USER	str	Usuario de PostgreSQL
POSTGRES_DB	str	Nombre de la base de datos
FIRST_SUPERUSER	EmailStr	Email del superusuario inicial
FIRST_SUPERUSER_PASSWORD	str	Contraseña del superusuario inicial

6.2. Variables opcionales con valores por defecto

Cuadro C.18: Variables opcionales con valores por defecto

Variable	Tipo	Default	Descripción
POSTGRES_PORT	int	5432	Puerto de PostgreSQL

Cuadro C.18: Variables opcionales con valores por defecto

Variable	Tipo	Default	Descripción
POSTGRES_PASS	str	“”	Contraseña de PostgreSQL
ACCESS_TOKEN	int	11520	Expiración del JWT (8 días)
FRONTEND_HOST	str	http://localhost:5173	URL del frontend para CORS
CORS_ORIGINS	list[str]	[]	Orígenes adicionales CORS
ENVIRONMENT	Literal	“local”	local staging production
SMTP_HOST	str	None	Host SMTP para envío de emails
SMTP_PORT	int	587	Puerto SMTP
SMTP_TLS	bool	True	Usar TLS en SMTP
SMTP_SSL	bool	False	Usar SSL en SMTP
SMTP_USER	str	None	Usuario SMTP
SMTP_PASSWORD	str	None	Contraseña SMTP
EMAILS_FROM	EmailStr	None	Email remitente
EMAIL_RESET_TOKEN	int	48	Expiración del token de reset
SENTRY_DSN	HttpUrl	None	DSN de Sentry para error tracking

6.3. Propiedades calculadas (no en .env)

Cuadro C.19: Propiedades calculadas (no en .env)

Propiedad	Descripción
SQLALCHEMY_DATABASE_URI	URI completa de conexión a PostgreSQL
all_cors_origins	BACKEND_CORS_ORIGINS + FRONTEND_HOST
emails_enabled	True si SMTP_HOST y EMAILS_FROM_EMAIL están configurados

7. Convenciones de Código

7.1. Backend (Python)

Orden de imports:

```
# 1. Biblioteca estándar
import uuid
from typing import Any
from decimal import Decimal

# 2. Terceros
from fastapi import APIRouter, HTTPException
from sqlmodel import SQLModel, Field

# 3. Locales
from app.api.deps import CurrentUser, SessionDep
from app.models import ProductCreate, ProductPublic
```



UNIVERSIDAD
SERGIO ARBOLEDA

Nomenclatura:

- Funciones y variables: `snake_case`
- Clases y modelos: `PascalCase`
- Modelos de BD: singular (`Product`, `Sale`)
- Schemas: `{Modelo}Create`, `{Modelo}Update`, `{Modelo}Public`, `{Modelo}sPublic`
- Aliases de tipo: `PascalCase` con `Annotated` (`CurrentUser`, `SessionDep`)

Anotaciones de tipo: obligatorias en todas las funciones. Usar `str | None` (no `Optional[str]`).

Patrón de rutas:

```
@router.post("/", response_model=ProductPublic)
def create_product(
    *,
    session: SessionDep,
    current_user: CurrentUser,
    current_organization: CurrentOrganization,
    product_in: ProductCreate,
) -> Any:
    product = Product.model_validate(product_in,
    ↪ update={"organization_id": current_organization})
    session.add(product)
    session.commit()
    session.refresh(product)
    return product
```

Códigos HTTP usados:

- 400: Solicitud inválida (datos incorrectos)
- 403: Sin permisos / credenciales inválidas
- 404: Recurso no encontrado
- 409: Conflicto (recurso duplicado)

Multi-tenancy: todas las consultas de negocio incluyen filtro `organization_id = current_organization`. El valor se extrae del usuario autenticado, nunca del cuerpo del request.

7.2. Frontend (TypeScript/React)

Orden de imports:

```
// 1. Externos
import { useState } from "react";
```

```
import { useMutation, useQueryClient } from "@tanstack/react-query";
import { z } from "zod";
```

```
// 2. Internos (alias @/)
```

```
import { ProductsService } from "@/client";
import { DataTable } from "@/components/Common/DataTable";
import { useCustomToast } from "@/hooks/useCustomToast";
```

Componentes: solo funcionales, un componente por archivo, nombre de archivo en PascalCase.

Patrón de mutación:

```
const mutation = useMutation({
  mutationFn: (data: ProductCreate) =>
    ProductsService.createProduct({ requestBody: data }),
  onSuccess: () => showSuccessToast("Producto creado"),
  onError: (err) => handleError(err),
  onSettled: () => queryClient.invalidateQueries({ queryKey:
    ↪ ["products"] }),
});
```

Validación de formularios: React Hook Form + Zod schemas con zodResolver.
No editar:

- src/client/**: auto-generado desde OpenAPI
- src/components/ui/**: componentes Shadcn/ui
- src/routeTree.gen.ts: auto-generado por TanStack Router

8. Pipeline de CI/CD

El repositorio incluye workflows de GitHub Actions:

8.1. Workflow de pruebas (en cada PR y push a main)

1. **Backend:** Ejecuta `uv run bash scripts/test.sh`; incluye Mypy, Ruff y Pytest con cobertura.
2. **Frontend:** Ejecuta `bun run lint`; Biome linting y verificación de tipos TypeScript.

8.2. Workflow de despliegue

Al hacer merge a main:

1. Construye las imágenes Docker de backend y frontend.



UNIVERSIDAD
SERGIO ARBOLEDA

9. Estructura de Pruebas

2. Las etiqueta con el hash del commit y con `latest`.
3. Las publica en el registro de imágenes.
4. Actualiza los contenedores en el servidor de producción via SSH.

9. Estructura de Pruebas

9.1. Backend (pytest)

```
backend/tests/
├── conftest.py                # Fixtures globales: db, client, auth headers
├── api/routes/
│   ├── test_login.py         # Autenticación, recuperación de contraseña
│   ├── test_users.py         # CRUD de usuarios, cambio de contraseña
│   ├── test_categories.py    # CRUD de categorías
│   ├── test_products.py     # CRUD de productos, ajuste de stock
│   ├── test_customers.py     # CRUD de clientes, historial de compras
│   ├── test_sales.py         # Registro y cancelación de ventas
│   ├── test_inventory_movements.py # Listado de movimientos
│   ├── test_dashboard.py     # Estadísticas del dashboard
│   └── test_private.py       # Endpoints de superusuario
└── crud/
    ├── test_user.py
    ├── test_category.py
    ├── test_product.py
    ├── test_customer.py
    ├── test_sale.py
    ├── test_inventory_movement.py
    └── test_dashboard.py
```

Fixtures disponibles en conftest.py:

Cuadro C.20: Backend (pytest)

Fixture	Descripción
db	Sesión de BD para pruebas (con rollback automático)
client	TestClient de FastAPI
superuser.token.headers	Headers con token del superusuario
normal.user.token.headers	Headers con token de usuario normal

Ejecutar pruebas:

```
cd backend
uv run bash scripts/test.sh          # Todos los tests con cobertura
uv run pytest tests/api/routes/ -v    # Solo tests de API
uv run pytest -k "test_create" -v    # Tests por patrón
```



9.2. Frontend (Playwright E2E)

```
frontend/tests/
├── auth.setup.ts           # Configuración de autenticación compartida
├── config.ts              # Variables de configuración de tests
├── login.spec.ts          # Flujo de inicio de sesión
├── sign-up.spec.ts        # Registro de usuario
├── organization-signup.spec.ts # Registro de organización
├── reset-password.spec.ts  # Recuperación de contraseña
├── inventory.spec.ts      # Módulo de inventario
├── sales.spec.ts          # Módulo de ventas
├── customers.spec.ts      # Módulo de clientes
├── admin.spec.ts          # Gestión de usuarios
├── user-settings.spec.ts  # Configuración de cuenta
└── utils/                # Helpers de pruebas E2E
```

10. Generación del Cliente API

El cliente TypeScript del frontend se genera automáticamente desde el esquema OpenAPI del backend:

```
# El backend debe estar corriendo en http://localhost:8000
cd frontend
bun run generate-client
```

La configuración del generador está en `frontend/openapi-ts.config.ts`. Los archivos generados en `src/client/` no deben editarse manualmente; cualquier cambio se sobrescribirá en la próxima generación.

11. Multi-Tenancy

OrbitEngine implementa multi-tenancy mediante **aislamiento por discriminador en tablas compartidas** (Shared Database, Separate Rows):

- Todas las tablas de negocio (`categories`, `products`, `customers`, `sales`, `sale_items`, `inventory_movements`) incluyen la columna `organization_id`.
- La `organization_id` se indexa en cada tabla para garantizar rendimiento en consultas filtradas.
- Toda consulta de datos de negocio filtra por `organization_id = current_user.organization_id`.
- El valor de `organization_id` **nunca** se acepta como parámetro del request; siempre se extrae del token JWT del usuario autenticado.
- Las restricciones de unicidad incluyen `organization_id` como parte de la clave compuesta (ej. `UNIQUE(organization_id, sku)`), permitiendo que el mismo SKU exista en organizaciones distintas.

11. Multi-Tenancy

Esta estrategia garantiza aislamiento completo de datos entre tenants sin necesidad de esquemas o bases de datos separadas, facilitando el mantenimiento y la escalabilidad horizontal.

Documento generado como parte del proyecto de grado. Universidad Sergio Arboleda, Semillero de Software como Innovación, Abril 2026.